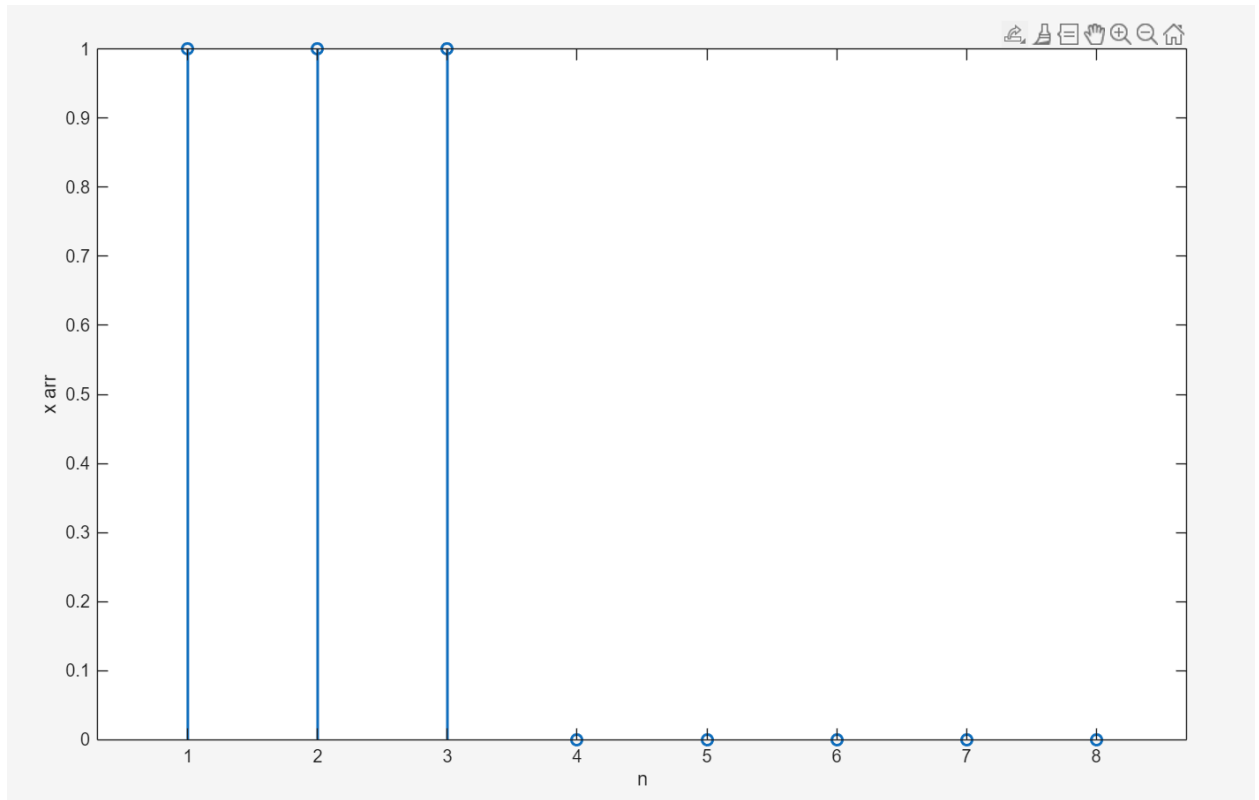


EE110B Lab#4

Nathan Taing

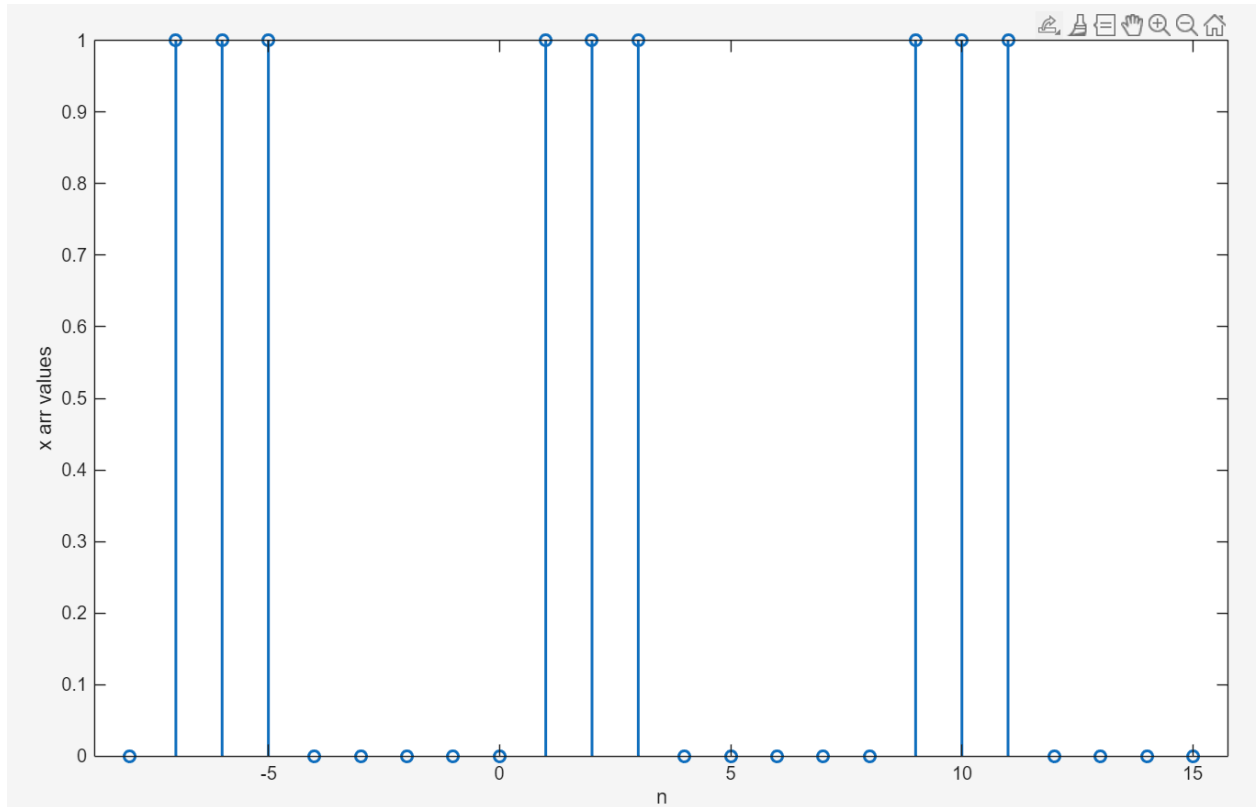
Implementing Periodic Convolution

a)



This plots the $x[n]$ according to the code provided.

b)

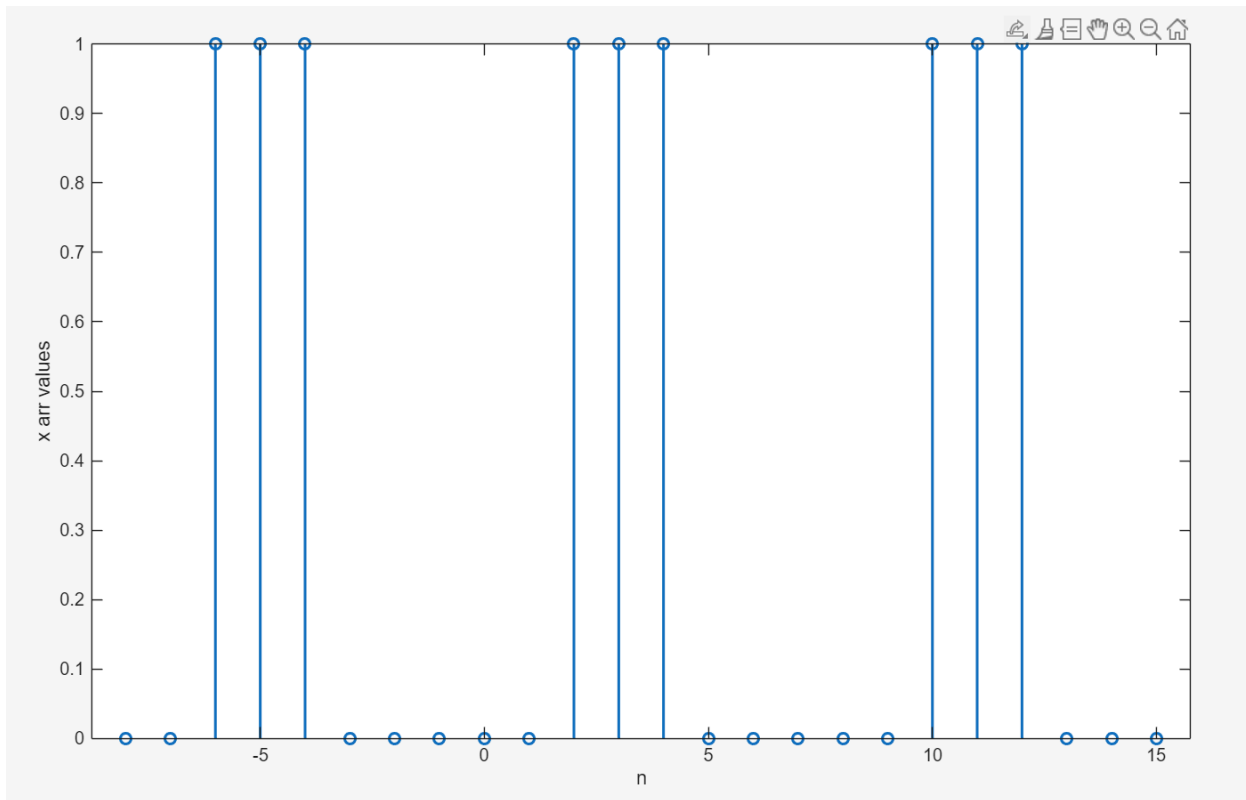


We plot the periodic signal from -8 to 15 above. This is based on the initial $x[n]$.

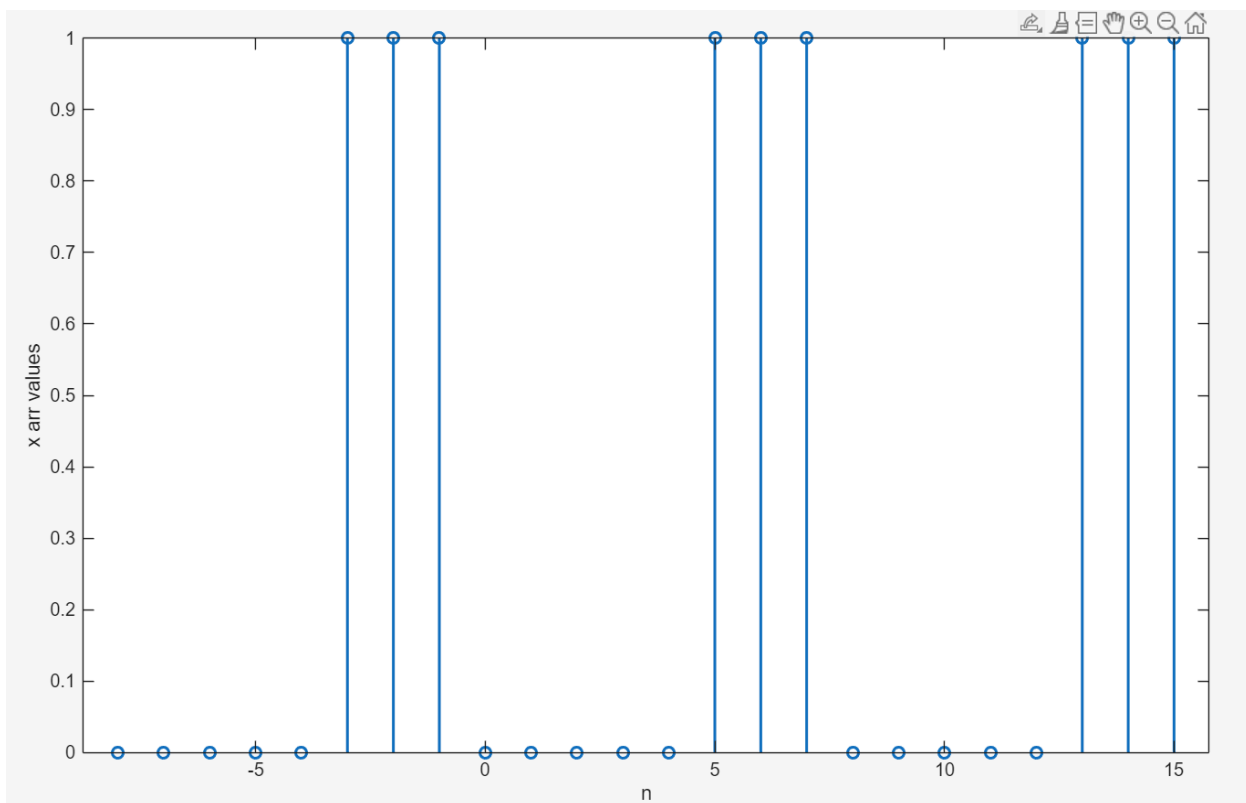
Code:

```
x_arr = zeros(1:8);
x_arr(1:3) = 1;
n = -8:15;
x_arr1 = zeros(size(n));
for i = 1:length(n)
    idx = mod(n(i), 8);
    if idx == 0
        idx = 8;
    end
    x_arr1(i) = x_arr(idx);
end
stem(n, x_arr1, 'LineWidth', 1.5);
xlabel('n');
ylabel('x arr values');
```

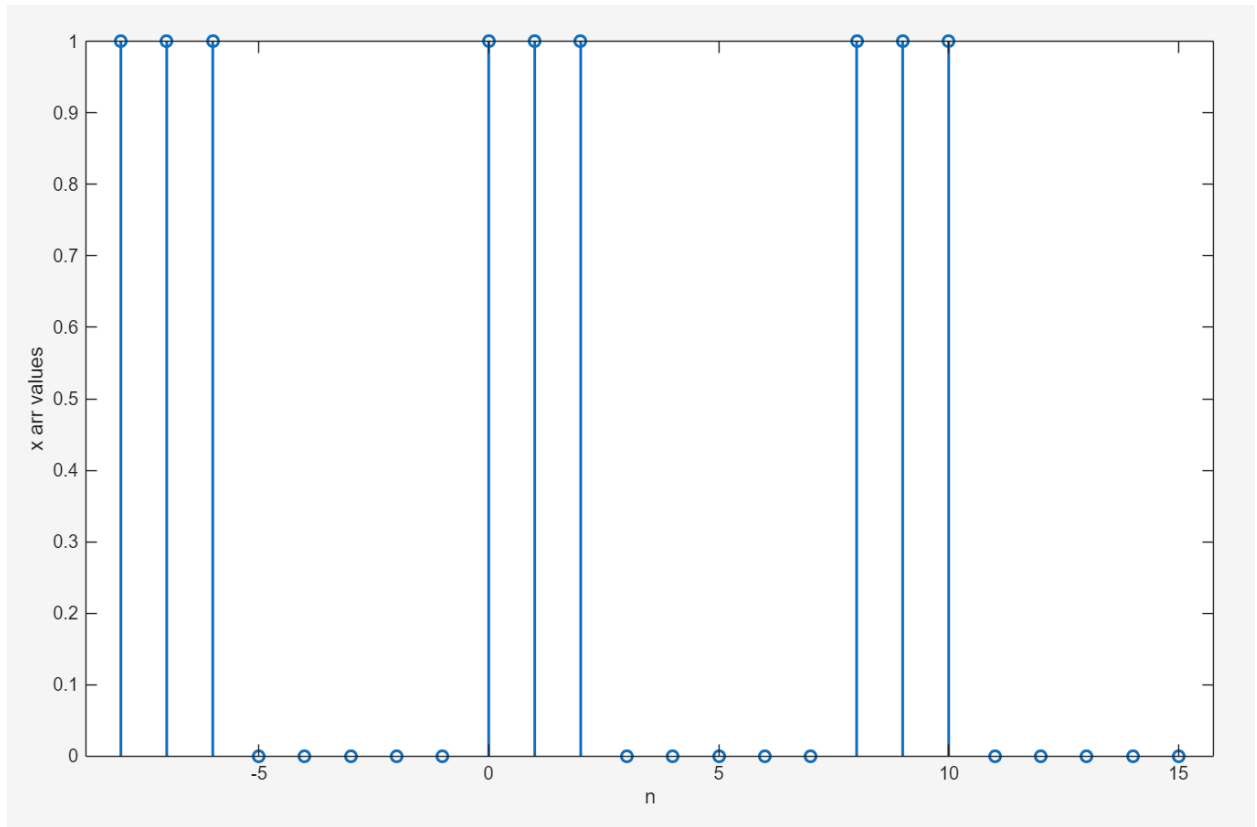
c) $x[n-1]$:



$x[n-4]$:



$x[n-7]$:



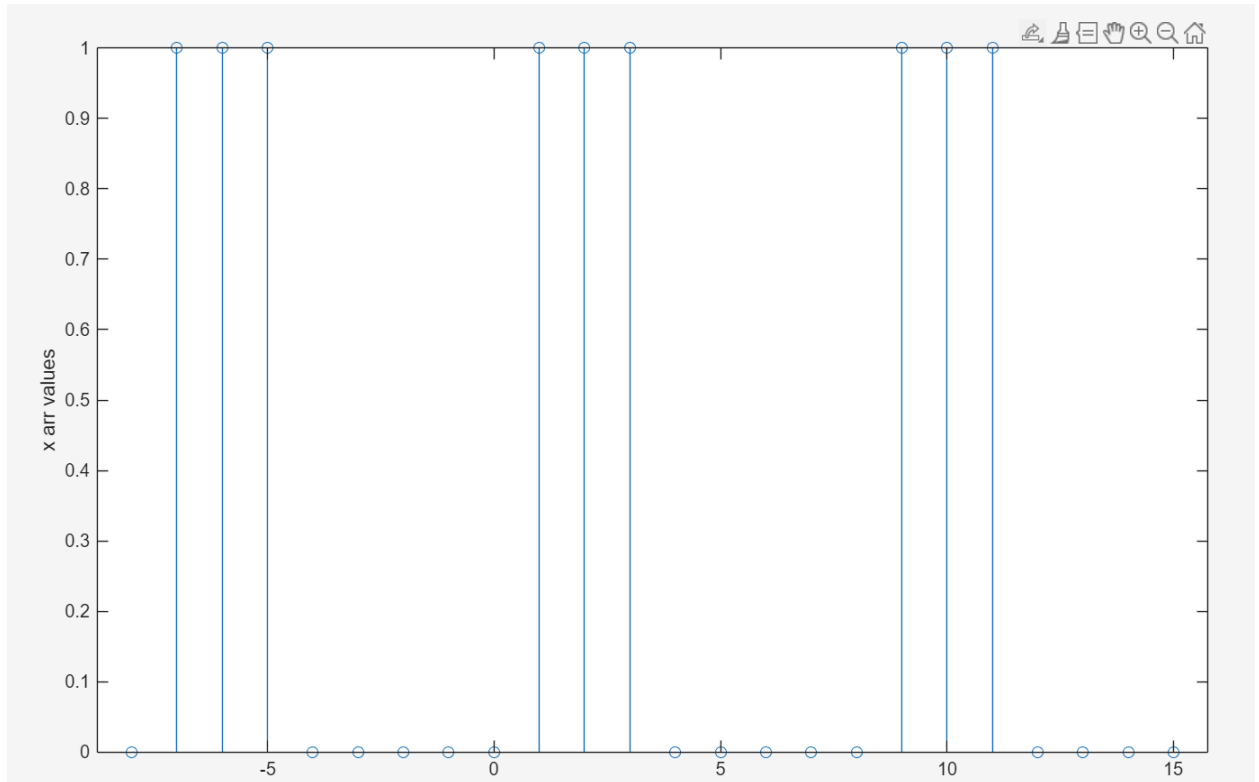
Each of the three signals plotted above represents their respective time delay. We can see each of the signals are time variations of the original signal provided in part a.

d) Code:

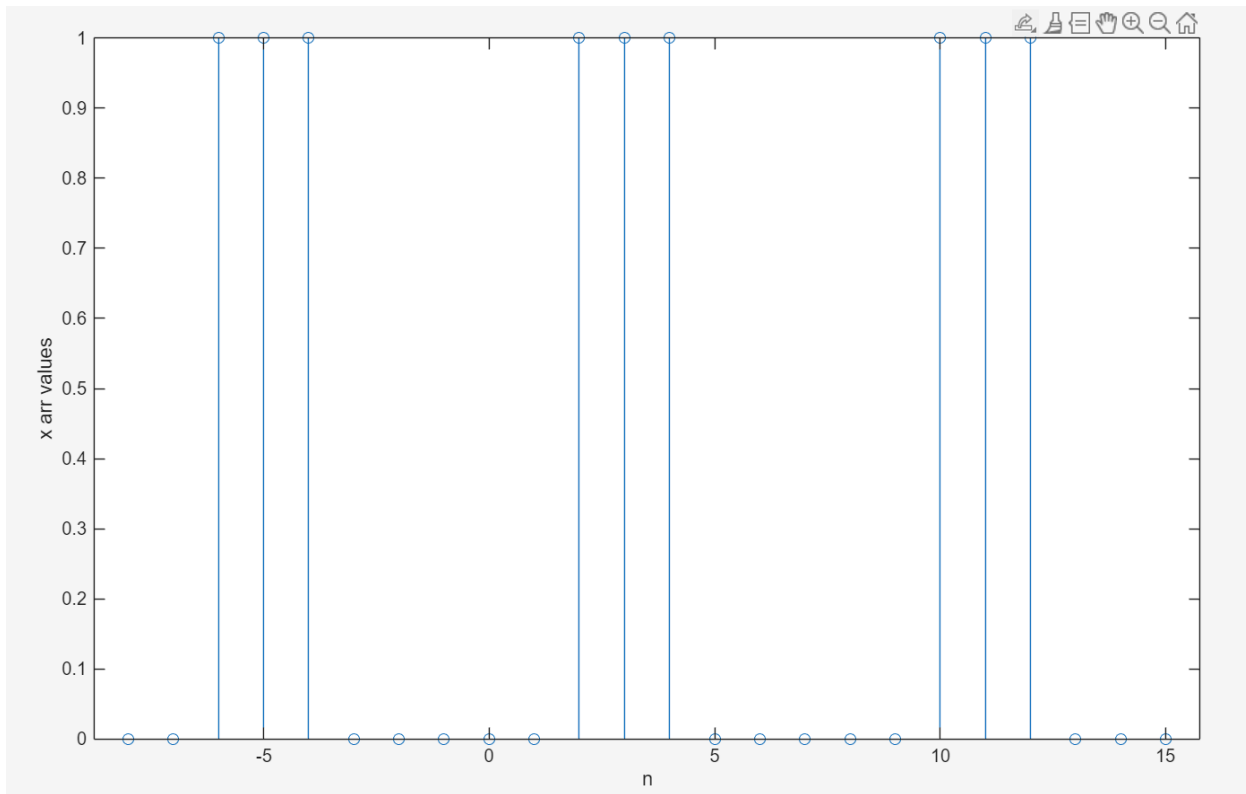
```
x_arr = zeros(1,8);  
x_arr(1:3) = 1;  
n = -8:15;  
x_arr1 = zeros(size(n));  
for i = 1:length(n)  
    idx = mod(n(i), 8);  
    if idx == 0  
        idx = 8;  
    end  
    x_arr1(i) = x_arr(idx);  
end  
figure  
n1 = 1:8;  
temp = circshift(x_arr1, 0);
```

```
stem(n, temp);  
xlabel('n');  
ylabel('x arr values');
```

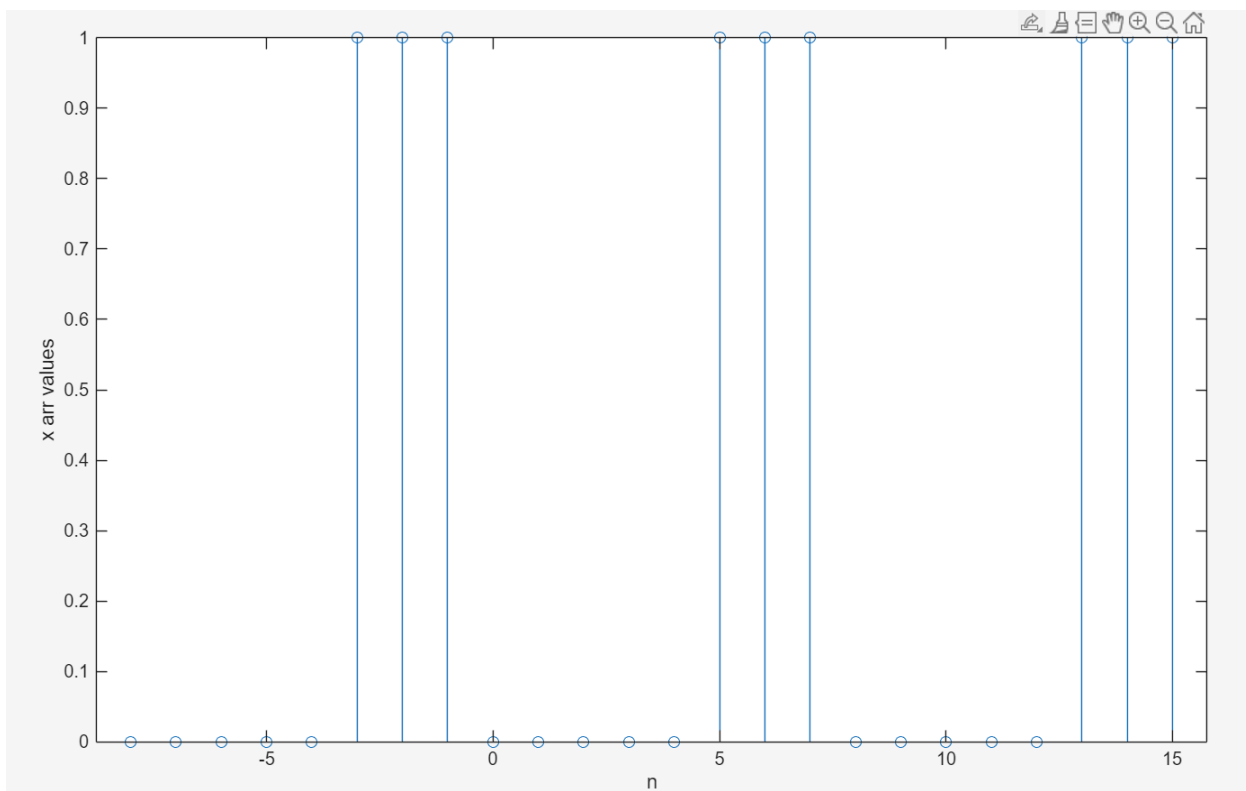
shift_val = 0



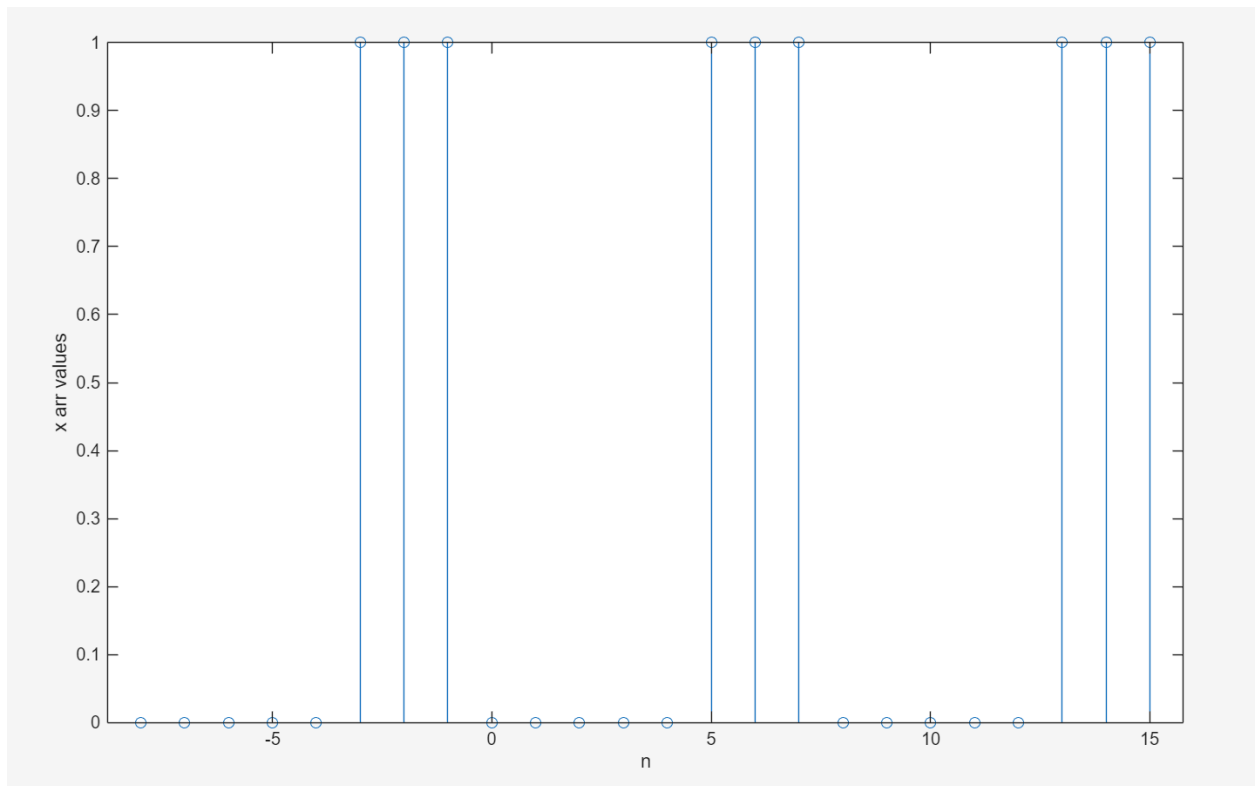
shift_val = 1:



shift_val = 4:

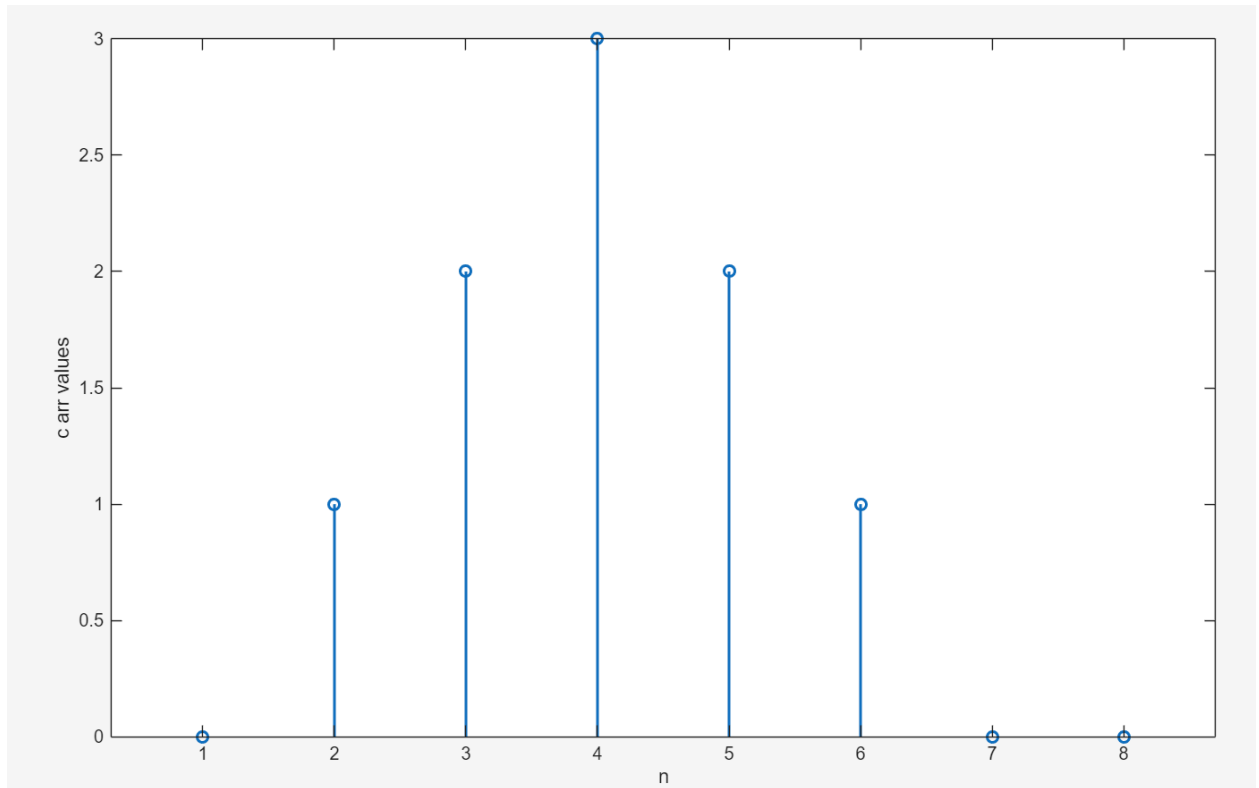


shift val = 7:



Yes, these plots are identical to the plots we found in part c. This would suggest `circshift` automatically does the time shift of a vector that we input.

e)



Code:

```
x_arr = zeros(1,8);
x_arr(1:3) = 1;
y_arr = zeros(1, 8);
y_arr(1:3) = 1;

c_arr = zeros(1, 8);
N = 8;
for idx = 1:N
    y_flipped = fliplr(y_arr);
    y_shifted = circshift(y_flipped, idx-1);
    c_arr(idx) = sum (x_arr.*y_shifted);
end
n = 1:length(c_arr);
stem(n, c_arr, 'LineWidth', 1.5);
```

Above, we plot the convolution of $x[n]$ and $y[n]$. We also provided the code of how to apply the convolution. This is a literal implementation of convolution using its equation.

f) Code:

```
a_k = fft(x_arr);
```

```
a_k =
```

```
Columns 1 through 5
```

```
3.0000 + 0.0000i 1.7071 - 1.7071i 0.0000 - 1.0000i 0.2929 + 0.2929i 1.0000 + 0.0000i
```

```
Columns 6 through 8
```

```
0.2929 - 0.2929i 0.0000 + 1.0000i 1.7071 + 1.7071i
```

```
b_k = fft(y_arr);
```

```
Columns 1 through 5
```

```
3.0000 + 0.0000i 1.7071 - 1.7071i 0.0000 - 1.0000i 0.2929 + 0.2929i 1.0000 + 0.0000i
```

```
Columns 6 through 8
```

```
0.2929 - 0.2929i 0.0000 + 1.0000i 1.7071 + 1.7071i
```

```
c_k = a_k.* b_k;
```

```
c_k =
```

```
Columns 1 through 5
```

```
9.0000 + 0.0000i 0.0000 - 5.8284i -1.0000 + 0.0000i 0.0000 + 0.1716i 1.0000 + 0.0000i
```

```
Columns 6 through 8
```

```
0.0000 - 0.1716i -1.0000 + 0.0000i 0.0000 + 5.8284i
```

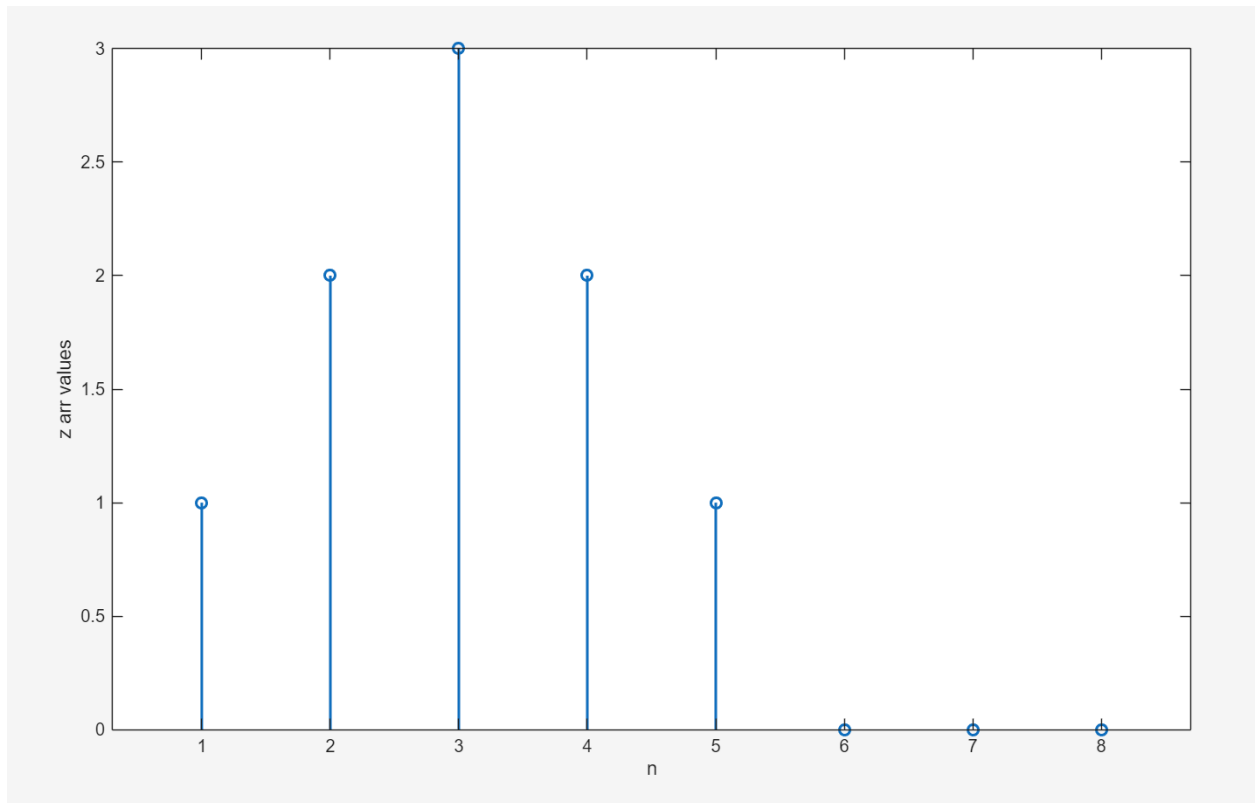
```
z_arr = ifft(c_k);
```

```
z_arr =
```

```
1 2 3 2 1 0 0 0
```

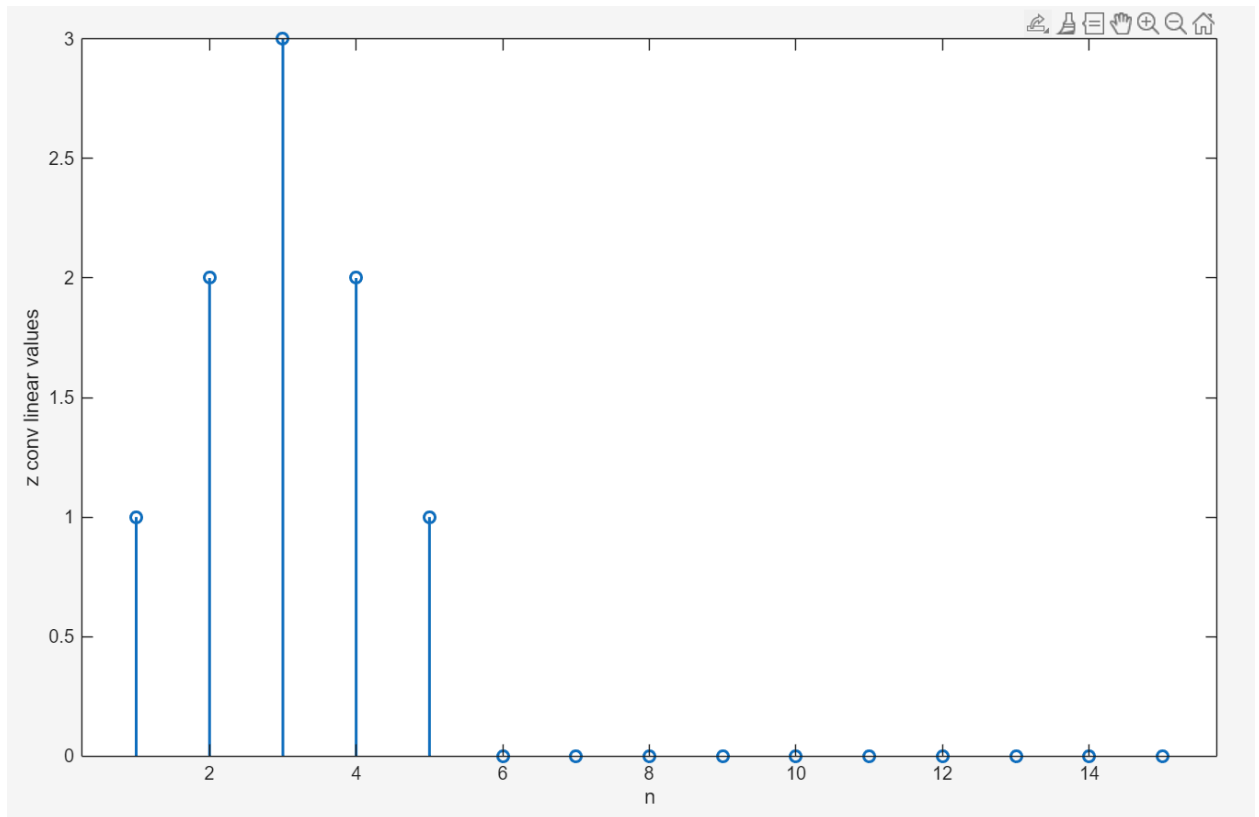
Each of the pictures above, provides the value of what was calculated using fft and ifft. These are the individual a_k , b_k , and c_k values calculated using the formula cited in the lab manual. These are the DTFS coefficients.

g)



Yes, the values found from z_arr do match the values we found using convolution. This is because periodic convolution in the time domain equals multiplication in the frequency domain. By finding the DTFS coefficients, we were working in the frequency domain. Therefore, we could multiply them together and inverse DTFS c_k (to get back into the time domain), and receive the same output as when we convolved $x[n]$ and $y[n]$.

h)

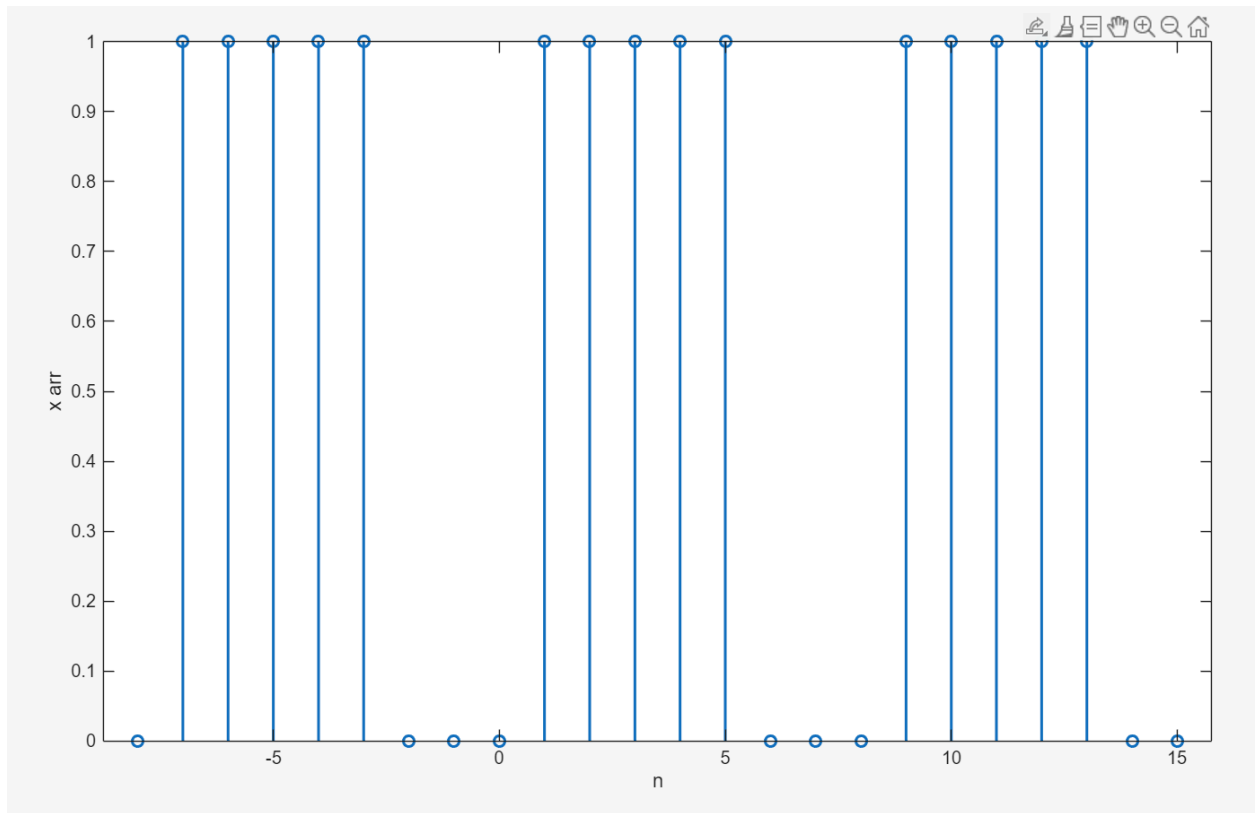


Code:

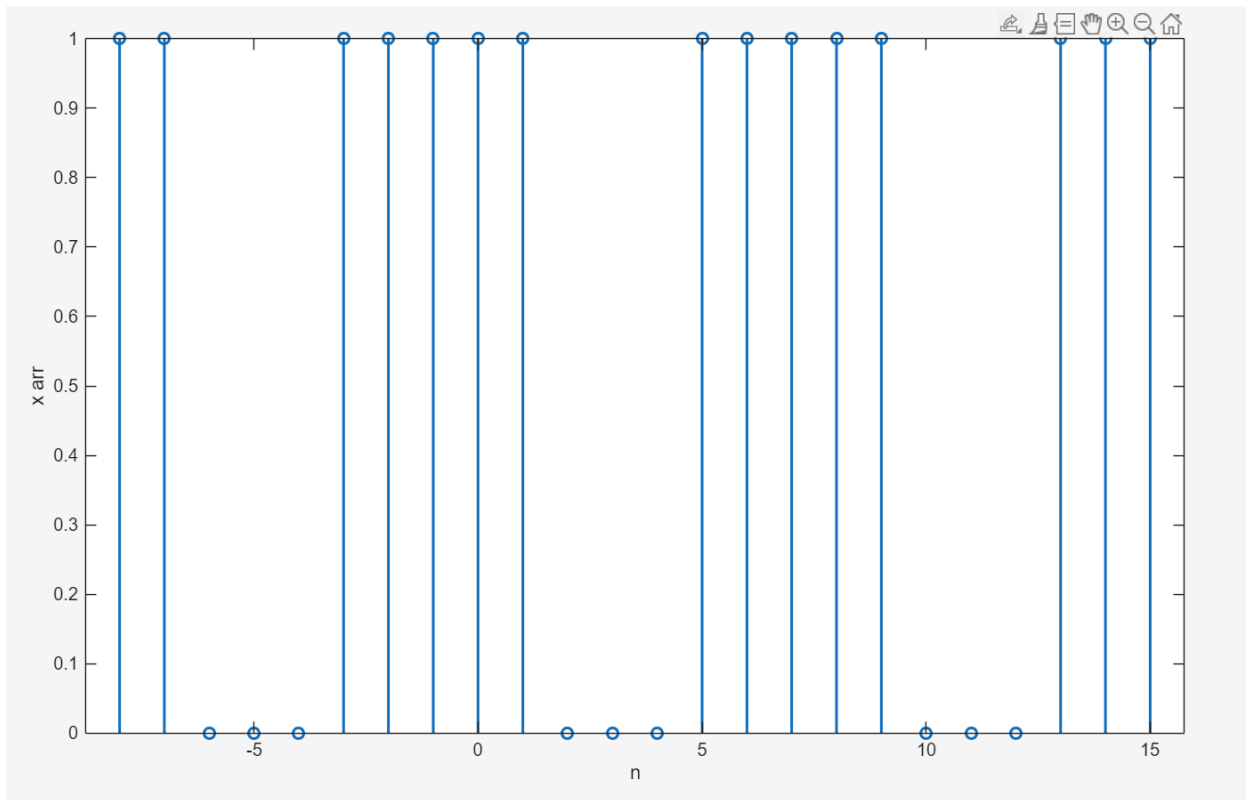
```
z_conv_linear = conv(x_arr, y_arr);
n = 1:length(z_conv_linear);
stem(n, z_conv_linear, 'LineWidth', 1.5);
xlabel('n');
ylabel('z conv linear values');
```

No this plot is not the exact same as the one we found in part e. While the shape of the signal is the same, we documented significantly more n values than before. This is because linear convolution includes the length of both signals and subtracts the total by 1. This results in the same convolution signal, but there is no wrapping, resulting in a longer signal. The periodic convolution assumes circular extension.

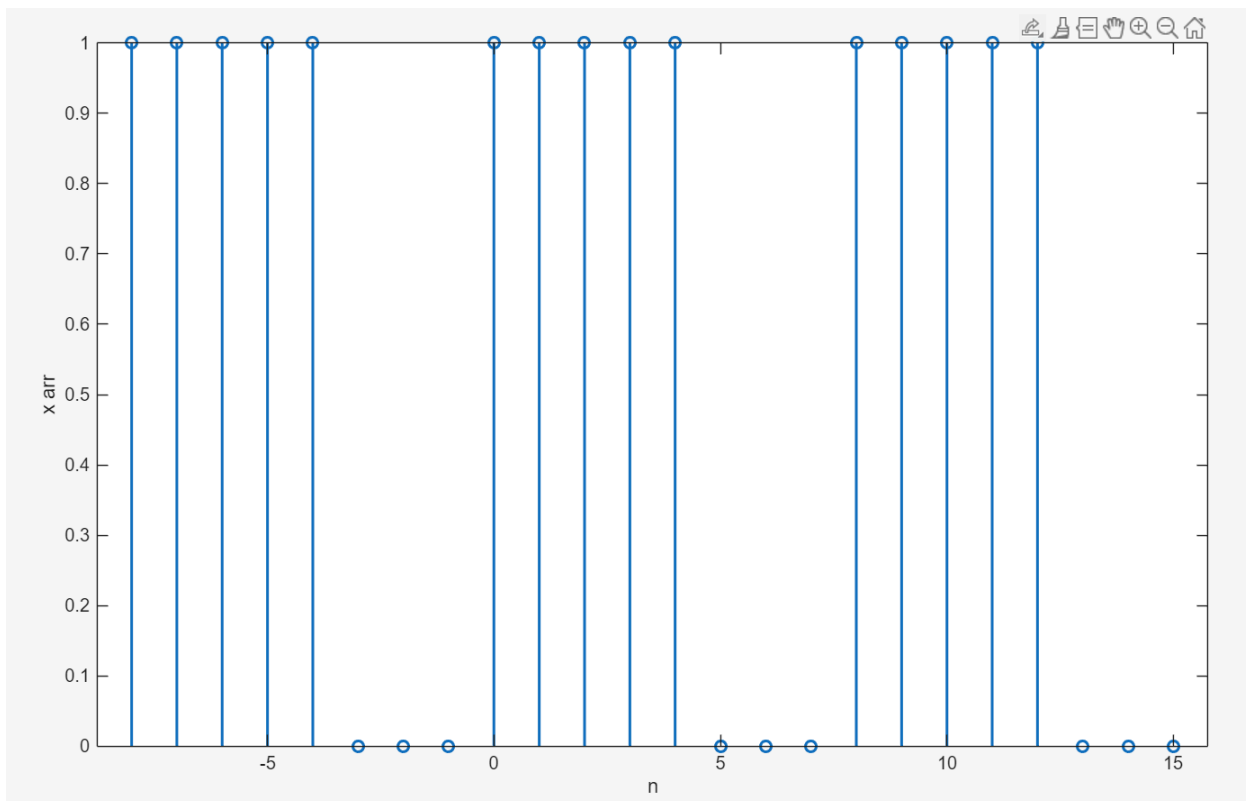
- i) Signal: $x[n] = [0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0]$
Part a & b)



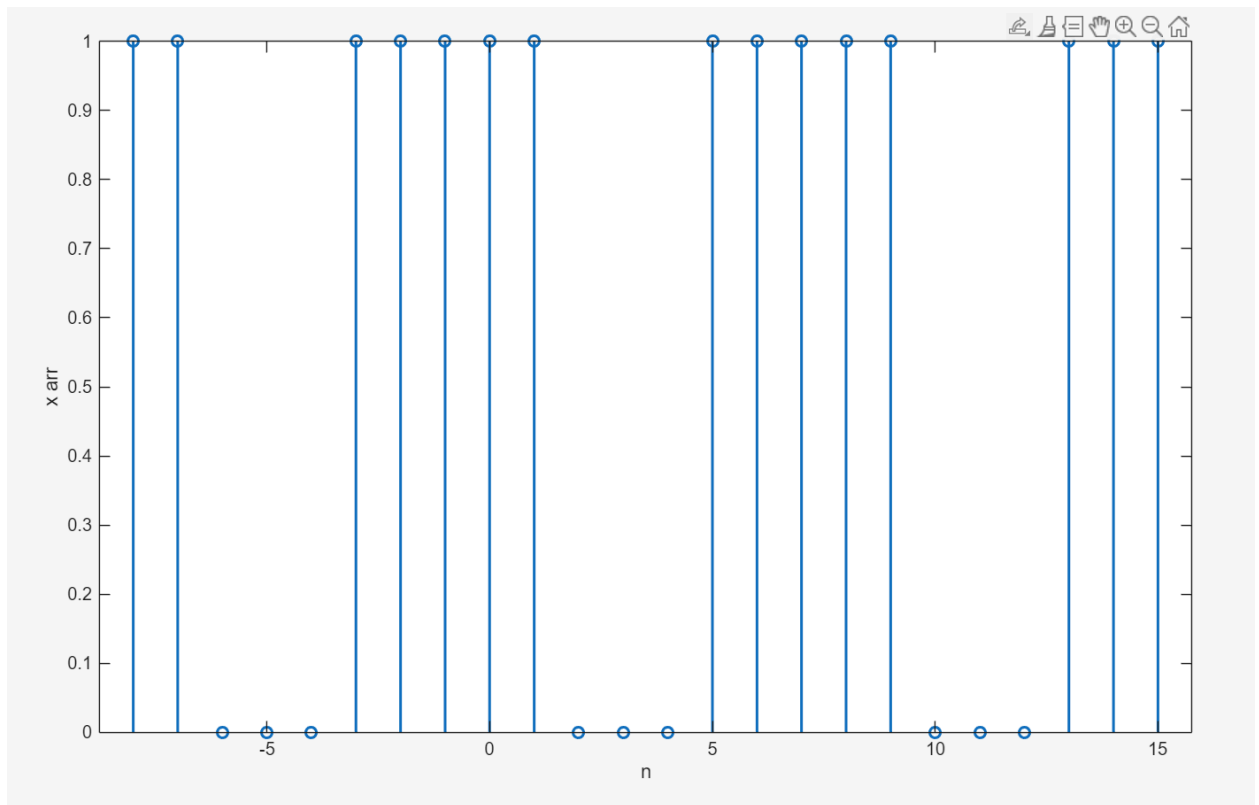
Part c)
 $x[n-4]$:



$x[n-7]$:

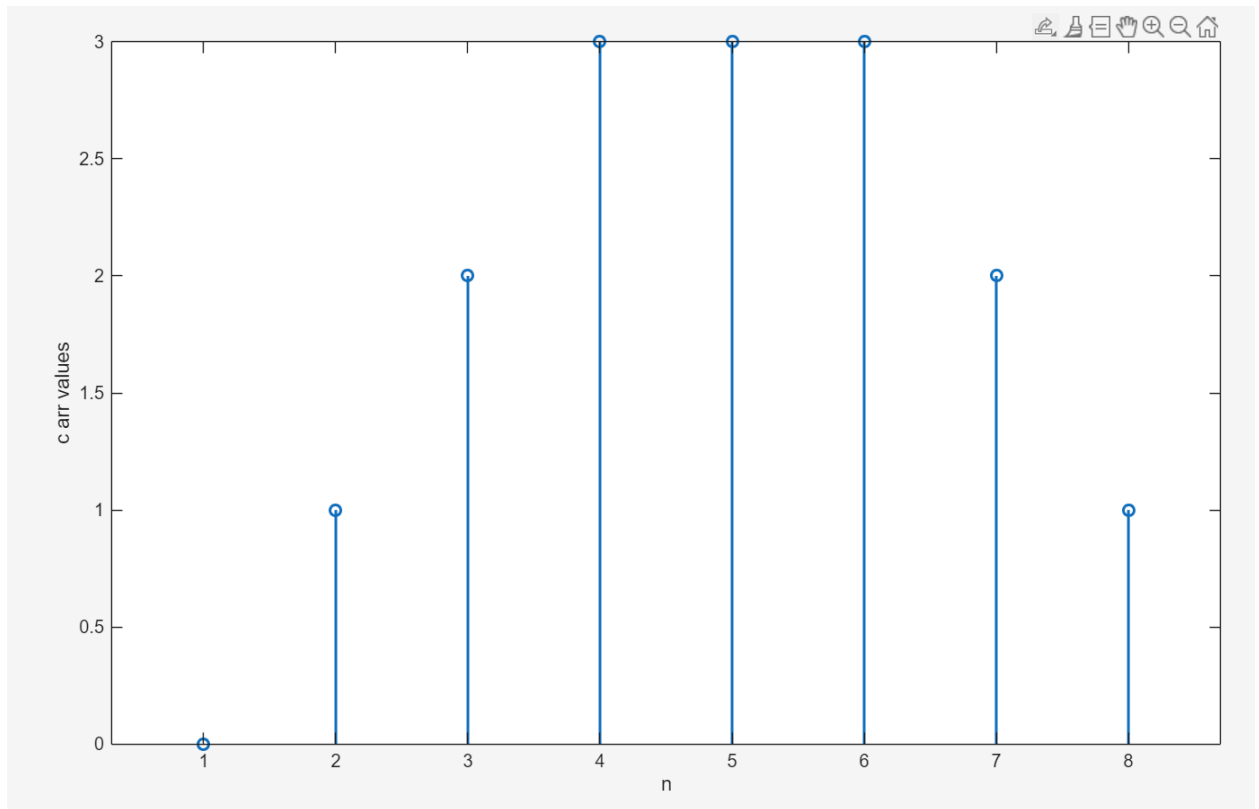


Part d)



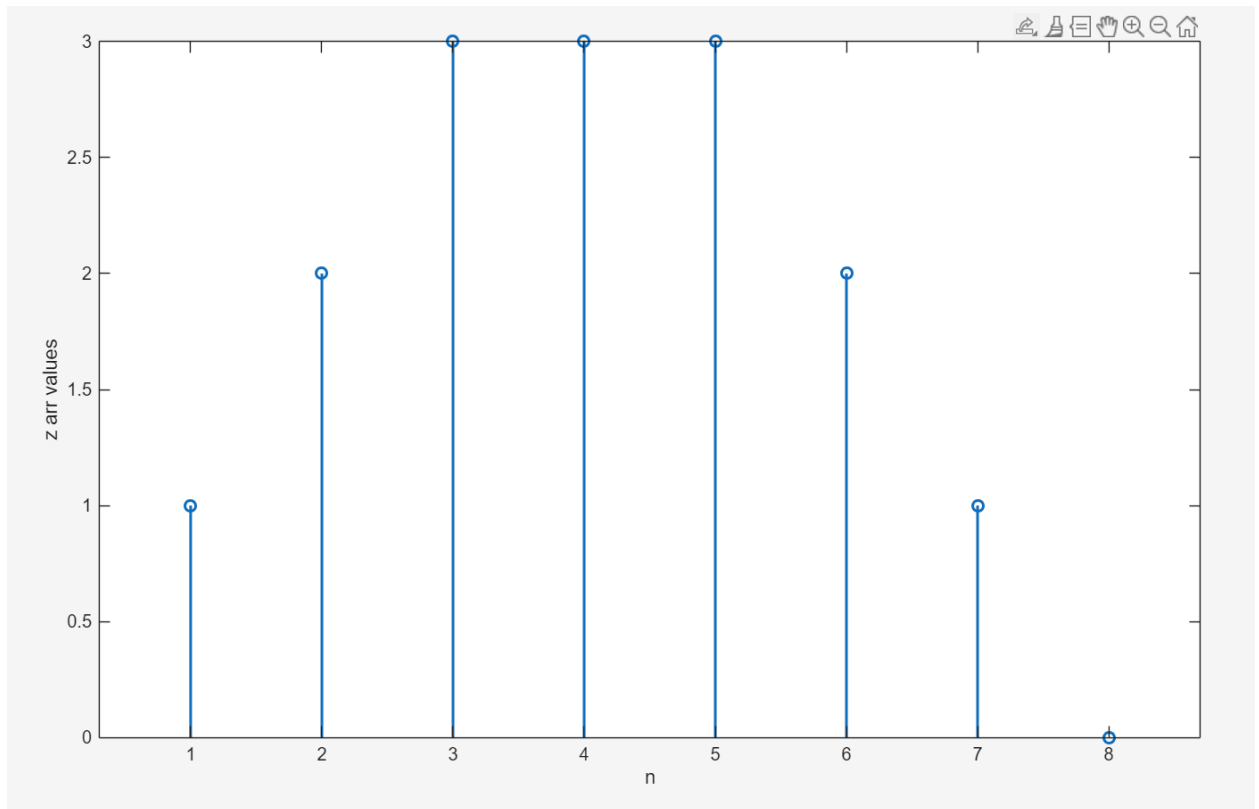
Yes, circshift still shifts the same amount as a time delay. This function still holds true in this new signal.

Part e)



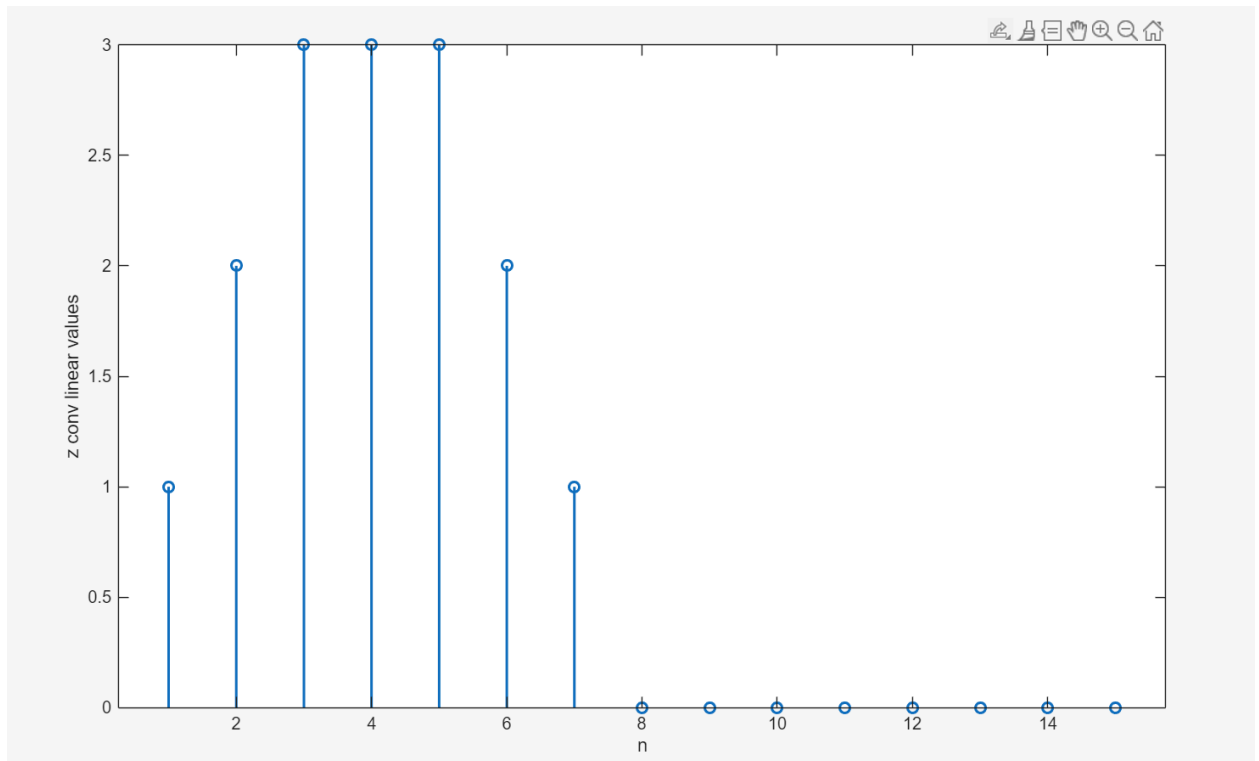
This picture shows the convolution result of the new $x[n]$ with the old $y[n]$.

Part f & g)



The plot is almost the same, except with one difference. The time is time shifted to the future by 1. The shape is the same, however, the first convolution plot has zero at 1 and the same sequence while this plot has the sequence and a zero at 8. This is most likely a calculation difference as fft utilizes the modulo operator, which can result in some indexing differences.

Part h)



The amount of ns that are recorded are much more, same as last instance. The shape of this graph is also the same as the one to part e. However, we are facing similar differences to part h, in that the indexing is different. This can be attributed to Matlab's built in formula differing from the equations we set in part e. Overall, while the shape is still the same, the overall placement is different, which can lead to different results and conclusions if not dealt with properly.